

Technical Documentation CLUE

Lukas Scholz
Humboldt-Universität zu Berlin
lukas.scholz@hu-berlin.de

Installation

First you need to clone the GitHub-Repository with the following command:

```
git clone https://github.com/lSch0lz/counterfactuals.git
cd counterfactuals
```

The project contains a `setup.py`, hence you can install all needed dependencies by running :

```
pip install -e .
```

Python Version

The Packages minimum requirement is Python 3.8

Project Structure

`counterfactual_xai/` contains the code for the package

`examples/` : contains several example usages e.g. MIMIC-LoS Regression

Inside the folder for the source code (`counterfactual_xai/`) you can find several sub-directories:

`methods/` : contains the code for generating CLUEs and several helper-functions

`utils/` : contains all necessary functions that are required for using CLUE (e.g. Training Scripts, DataLoaders)

Code differences

Due to the re-implementation in Python3 the code differs here the most from the original repository

Prerequisites for using CLUE

There are several steps you have to do before you are able to generate CLUEs.

The following description of those steps is based on the MIMIC-LoS Regression.

Bayesian Neural Network

The Bayesian Neural Network is used to generate the uncertainty estimates:

$$\mathcal{H}(y|\mu_{\theta}(x|z))$$

The network outputs two variables. μ represents the mean and σ is the standard deviation of the predictive distribution. When using the model to predict a certain value you can use μ as the predicted value and σ as the uncertainty estimation for this prediction.

Train a Bayesian Neural Network

You can find the script for starting the training under: [counterfactual_xai/utis/clue/bnn/train_script.py](#)

✓ Todo

To be able to run this script you need to change the path to your data-file and the path where your model should be saved to

After creating a train-test split:

```
x_train, x_test, y_train, y_test = train_test_split(features, LOS, test_size=.20,
random_state=0)
```

You need to create a instance of the `DataFeed()` class for both train- and valset:

```
trainset = DataFeed(x_train, y_train, transform=None)
valset = DataFeed(x_test, y_test, transform=None)
```

After doing so there are several hyperparameters, which are all copied from the original CLUE repository.

At first you need to initialise the `GaussianMLP()` class:

```
model = GaussianMLP(input_dim, width, depth, output_dim, flatten_image=False)
```

which is later used by the actual `GaussianBNN()`:

```
net = GaussianBNN(model, N_train, lr=lr, cuda=cuda)
```

Now you are ready to go! The function `train_BNN_regression()` contains the training-loop and data-loading functionalities needed.

⚠ CLUE Classification

Even though there exists a training script for classification, the library currently only works for regression

Predicting with a Bayesian Neural Network

You can find a evaluation script, where we do single predictions under:

`counterfactuals/examples/evaluate_bnn_regression.py`

To use the trained BNN for inference you first need to load the model:

```
input_dim = x_train.shape[1]
width = 200
depth = 2
output_dim = 1
N_train = x_train.shape[0]

mlp_gauss = GaussianMLP(input_dim, width, depth, output_dim, flatten_image=False)

bnn_gauss = GaussianBNN(mlp_gauss, N_train, lr=LEARNING_RATE, cuda=False)
bnn_gauss.load_weights(SAVE_DIR)
```

After that you can predict values:

```
mu, sigma = bnn_gauss.predict(val)
mu_denormalized = mu.detach().numpy() * y_stds + y_means
```

`y_stds` and `y_means` are obtained through the training dataset.

Info

It is always necessary to denormalize the output of the prediction

Deep Generational Model

CLUE uses a Variational Auto-Encoder as the Deep Generative Model. The VAE is used to search for a point inside the latent-space, that is more certain, while being close to the input point x_0 .

Train a Variational Autoencoder

You can find the script for starting the training under: [counterfactual_xai/utis/clue/vae/train_script.py](#)

Todo

To be able to run this script you need to change the path to your data-file and the path where your model should be saved to

The train script for the VAE is very similar to the train script for the BNN. The major difference is how the Input-Dimensions are constructed.

For the BNN:

```
input_dim: int = x_train.shape[1]
```

For the VAE:

```
input_dim_vec: List[int] = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 29]
```

As you can see the Input-Dimensions here are constructed out of a list of integer values.

You can interpret the list as the following:

- All continuous variables are encoded as a 1
- All categorical variables are summed up and encoded as the sum

For our LoS Regression Dataset there are 20 continuous and 29 categorical variables. Therefore the list contains 20 ones and a 29 as such.

Latent Dimensions

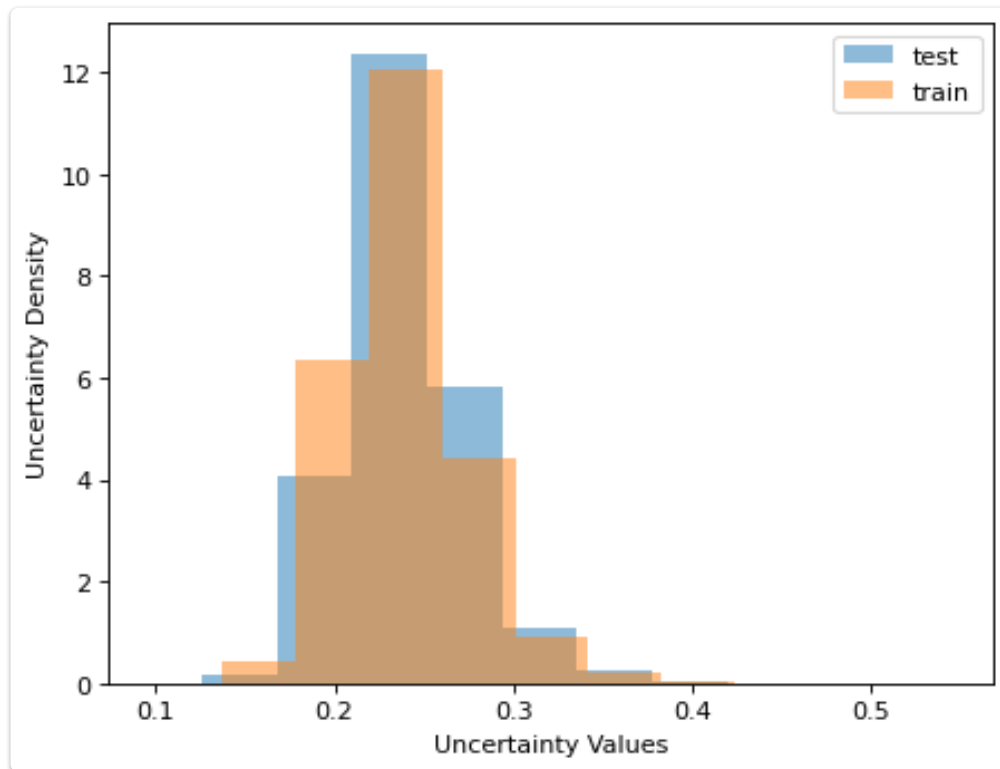
The VAE training script contains a loop for different latent dimensions. For each dimension there will be a single `theta_best.dat` and `theta_last.dat` model file, which contains the weights of the trained model. The used latent dimensions are copied from the original CLUE repository.

Visualisations

The following section explains the visualisations provided inside the [mimic_clue_regression.ipynb](#) Notebook.

Train / Test Uncertainty

The first plot shows the uncertainty distribution inside your train- and test dataset.

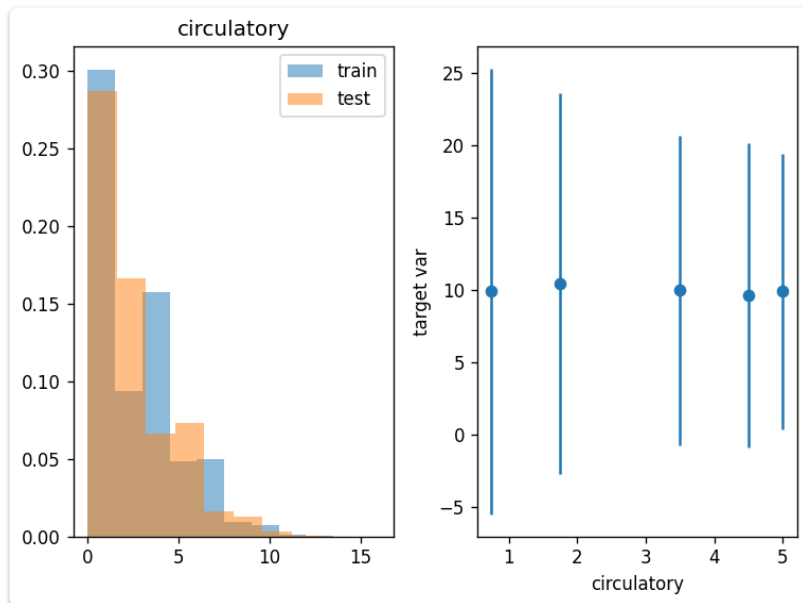


As you can see here both train- and test dataset seems to have a similar uncertainty distributions.

Uncertainties per Feature

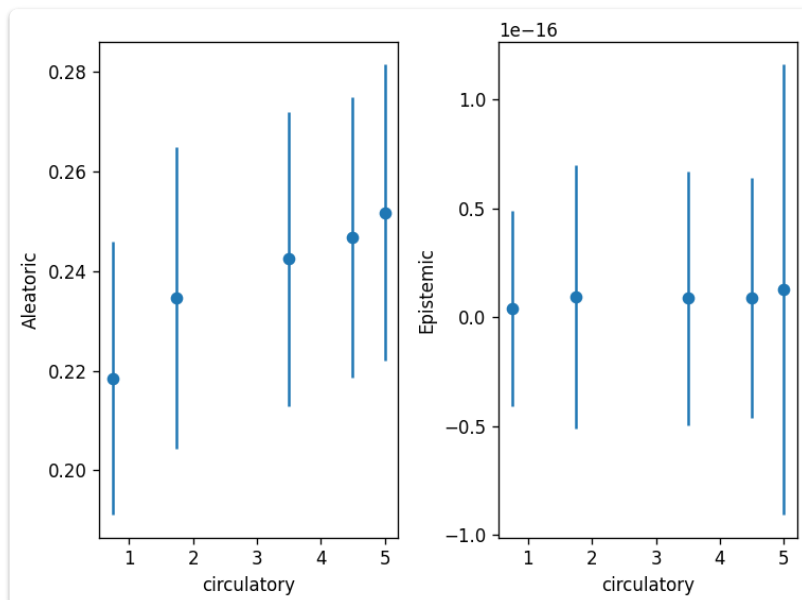
The next few plots can be generated per Feature by adjusting the parameter `varn_N` inside the Notebook-Cell.

The first plot is showing the uncertainty distributions for test- and train dataset for this particular feature and how the target variable (here Length-of-Stay) corresponds to the value of the chosen feature.



As you can see the uncertainty for this feature is, compared to the whole dataset, relatively low. The right plot show us, that the more circulatory ICD-9 codes where found, the less variance there is inside the LoS-values. Also the mean value is always at the same level (around 10 days).

The next plot shows the aleatoric and epistemic uncertainty for the chosen feature.

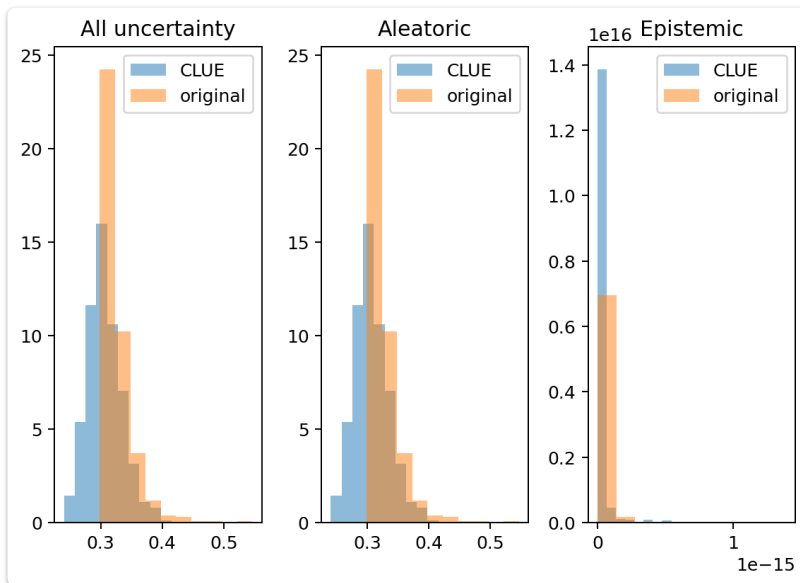


If more circulatory diseases are diagnosed, the higher the aleatoric uncertainty inside the data is. The epistemic uncertainty is at a fairly same level, there is no real trend to observe.

CLUE Plots

The next plots are directly related to CLUE.

At first there is a comparison between the uncertainty values calculate before applying CLUE and after.

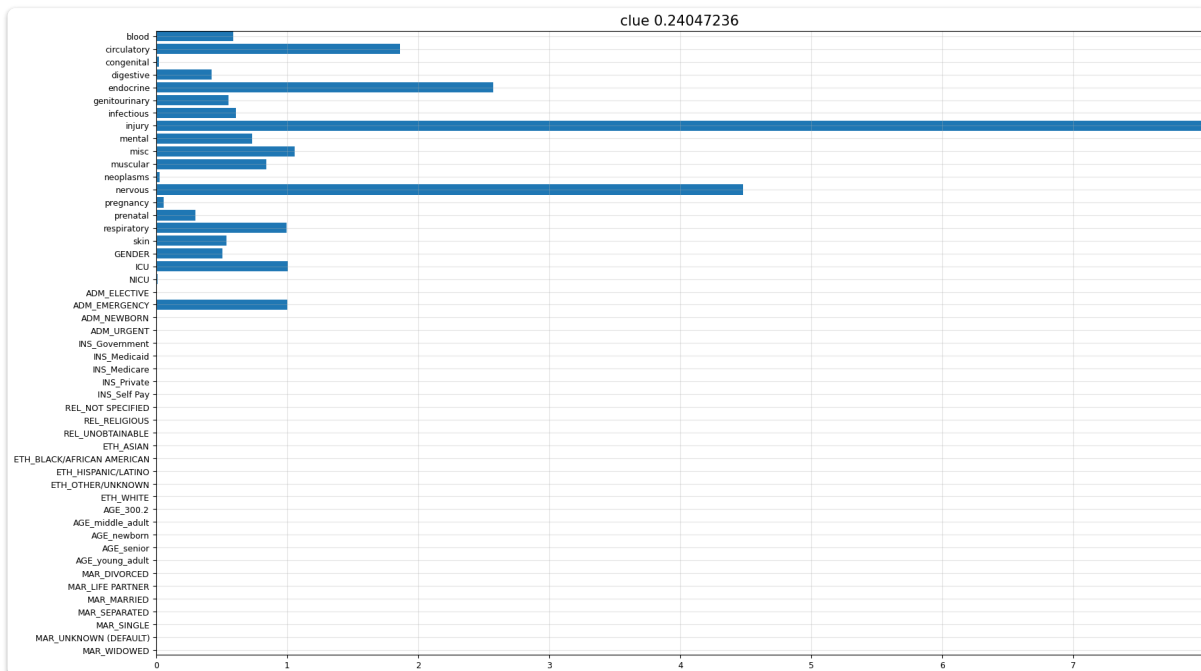


This shows, that after applying CLUE the aleatoric, epistemic and total (all) uncertainty distribution shifted to the left, hence became more certain.

The total uncertainty is calculated by

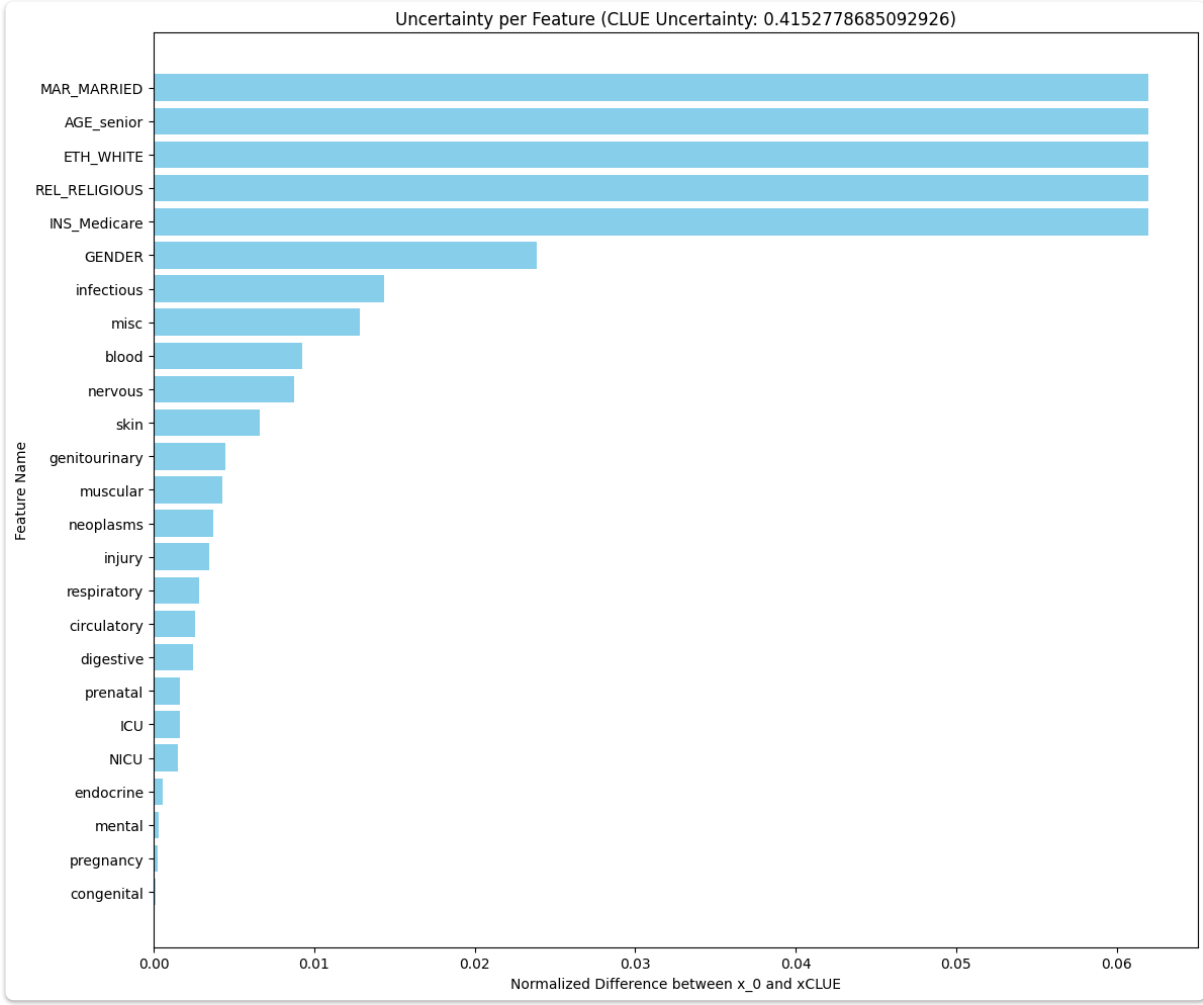
$$Uncertainty = Epistemic + Aleatoric$$

The next plot is related to the x_{CLUE} , which is the result of the optimisation inside the latent-space.



The bars refer to the new value each feature was assigned, after the optimisation process. Before optimising the input x_0 the patient might be diagnosed with 6 different circulatory diseases, but CLUE found, that the prediction would be more certain, if only 2 diagnoses would be there. This new value is shown inside this plot.

Coming up is a plot, that was created to show the difference between the original input x_0 and the generated, more certain x_{CLUE} .



The idea behind this plot is to show the features which contribute most to the overall uncertainty. To calculate the values, each feature is first normalised (Min-Max Normalisation):

$$\text{Normalized Difference} = \frac{x_{CLUE,i} - \min(x_i)}{\max(x_i) - \min(x_i)} - \frac{x_{0,i} - \min(x_i)}{\max(x_i) - \min(x_i)}$$

where $x_{CLUE,i}$ is the CLUE value for feature i , $x_{0,i}$ is the baseline value for feature i , and x_i represents all values of feature i .

Then, the normalised difference is scaled with respect to the total model uncertainty:

$$\text{Scaling Factor} = \frac{\sigma_{\text{total}}}{\sum_{i=1}^n \sigma_i}$$

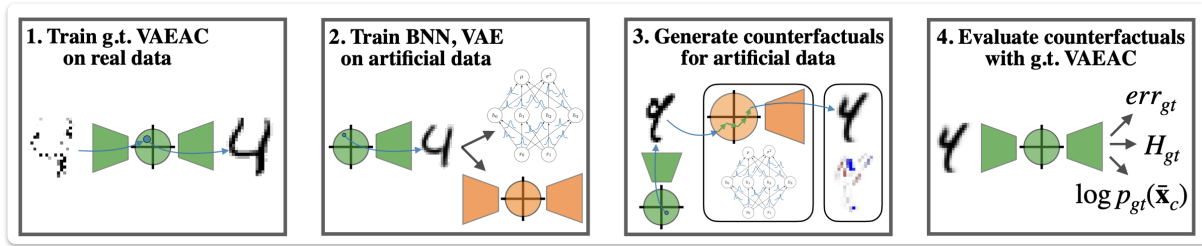
$$\text{Scaled Uncertainty for Feature } i = (\text{Normalized Difference})_i \times \text{Scaling Factor}$$

where σ_{total} is the overall model uncertainty, σ_i is the uncertainty contribution of feature i , and n is the total number of features.

Here lays also future work on how to make sure, that the categorical change from 0 to 1, which represents a 100% difference, doesn't negatively influence the calculation of the feature-based uncertainty estimation.

Evaluation Framework

Since the evaluation framework is currently **NOT** tested you can find it on the feature branch [/xclue-differences](#). On this feature branch, the experimental uncertainty per feature element is implemented.



The evaluation framework uses an additional VAE that produces artificial data that is used to evaluate the quality of generating counterfactual explanations.